

Video Display Processors

TMS9918B Programmer's Guide

Supplement

Unlocking and detecting the TMS9918B
Palettes, extended modes and table layouts
Two-bit patterns, fat dots, bitmap and 64-column text
Sprite pairs, scrolling and scanline interrupts
Z80 examples for SC-3000 class systems

ADVANCE INFORMATION

Hypothetical 1983 device - design study prepared for the GearSF7000 emulator.
Not a Texas Instruments product. Companion to the TMS9918B Data Manual (TB-9918B-01).

September 2026

IMPORTANT NOTICE

This supplement describes the programming of the TMS9918B, a hypothetical revision of the TMS9918A developed as a design study. It assumes the reader knows the Video Display Processors Programmer's Guide (SPPU004) and covers only what the TMS9918B adds. Electrical and timing data are in the TMS9918B Data Manual.

Copyright 2026 Saverio Russo. This document is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). Anyone may implement the device it describes, in hardware or software, and may copy and adapt the document, provided that credit is given. Implementations may call themselves TMS9918B compatible when they meet the conditions of COMPATIBILITY.md.

Bit 0 is the MSB and bit 7 the LSB, as in SPPU004. Z80 examples use the SC-3000 ports: data BEh (MODE = 0) and control BFh (MODE = 1). The example programs may also be used under the Apache License 2.0.

TABLE OF CONTENTS

1. INTRODUCTION	1-1
1.1 General TMS9918B Operation	1-1
1.2 Reference Material	1-1
2. FEATURES	2-1
2.1 Display Planes	2-1
2.2 Extended Display Modes	2-1
2.3 Available Colours	2-1
2.4 Palettes	2-1
3. TALKING TO THE TMS9918B	3-1
3.1 Unlocking the Extended Registers	3-1
3.2 Detecting the TMS9918B	3-1
3.3 Register Aliasing While Locked	3-2
3.4 Writing the Palette	3-2
3.5 Reading the Status Registers	3-3
4. DESCRIPTION OF THE EXTENDED REGISTERS	4-1
4.1 Extended Write-Only Registers	4-1
4.2 Status Register S1	4-2
5. INITIALIZING THE EXTENDED MODES	5-1
5.1 Graphics1X Mode Initialization	5-1
5.2 Graphics2Fat Mode Initialization	5-1
5.3 Bitmap Mode Initialization	5-2
5.4 Text40X and Text64 Mode Initialization	5-3
6. CREATING PATTERNS FOR EXTENDED MODES	6-1
6.1 Two-Bit Tile Patterns	6-1
6.2 Graphics2Fat Patterns	6-1
6.3 Text Patterns	6-1
7. THE EXTENDED DISPLAY MODES	7-1
7.1 Graphics1X Mode	7-1
7.2 Graphics2Fat Mode	7-1
7.3 Bitmap and BitmapQ Modes	7-1
7.4 Text40X Mode	7-1
7.5 Text64 Mode	7-1
8. SPRITES IN EXTENDED MODES	8-1
8.1 Sprite Colour	8-1
8.2 BANK	8-1
8.3 Sprite Pairs	8-1
8.4 Sprites per Line	8-1
8.5 Sprites in the Text Modes	8-1
9. SCROLLING	9-1
9.1 The World and the Screen	9-1
9.2 What the VDP Does With Them	9-1
9.3 MASK, HLOCK and VLOCK	9-1
9.4 When a Written Value Becomes Visible	9-1
9.5 Scrolling Beyond One Screen	9-2
9.6 Sprites Are Not Scrolled	9-2
10. THE SCANLINE INTERRUPT	10-1
10.1 The Line Counter	10-1
10.2 Enabling and Acknowledging	10-1
10.3 What a Handler Can Change in Time	10-1

10.4 Four Things a Band Can Do	10-2
11. OTHER PROGRAMMING TIPS	11-1
11.1 Palette Effects	11-1
11.2 Loading VRAM Quickly	11-1
APPENDIX A - REGISTER MAP	A-1
APPENDIX B - CPU TO VDP ACCESS TIMES	B-1
APPENDIX C - ADDRESS LOCATION TABLES	C-1
APPENDIX D - Z80 SUPPORT ROUTINES	D-1

1. INTRODUCTION

This is a supplement to the TI Video Display Processors Programmer's Guide. Everything that guide says about the TMS9918A remains true for the TMS9918B: the same ports, registers, tables and display modes are available after power-up, and existing software runs unchanged. This supplement explains how to unlock and use the extended functions: palettes, extended display modes, eight sprites per line with sprite pairs, hardware scrolling and the scanline interrupt.

The programming examples are written in Z80 assembly language for SC-3000 class systems. The routines of Appendix D are used by all examples.

1.1 General TMS9918B Operation

The TMS9918B fetches data from VRAM, processes it into a serial stream of pixels and gives the CPU access to its registers and to VRAM between fetches, exactly like the TMS9918A. The difference is inside the loop: memory cycles are shorter, adjacent bytes are fetched together, and in extended modes the VDP fetches two-byte entries (for example a pattern name and its attribute) in a single memory cycle. For the programmer this means that extended tables are made of byte pairs or groups stored next to each other.

1.2 Reference Material

Document	Content
TMS9918A/28A/29A Video Display Processors Data Manual (MP010A)	TMS9918A hardware
Video Display Processors Programmer's Guide (SPPU004)	TMS9918A programming
TMS9918B/9928B/9929B Data Manual (TB-9918B-01)	TMS9918B hardware and timing
TMS9918B Design Notes	Design rationale and history

2. FEATURES

2.1 Display Planes

The 35 display planes of the TMS9918A remain: 32 sprite planes, the pattern plane, the backdrop and the external VDP plane. In Graphics1X and Graphics2Fat a tile can be placed in front of the sprite planes with its PRIOR attribute bit; the backdrop still shows where the tile pixel value is 0.

2.2 Extended Display Modes

Mode	Screen	Colours	Sprites/line	Typical use
Graphics1X	32 x 24 tiles, 512 patterns	3 + backdrop per tile, 4 palettes	8	games with scrolling playfields
Graphics2Fat	128 x 192 dots, 768 patterns	15 per dot	8	title screens, drawing programs
Bitmap / BitmapQ	256 x 192 pixels	3 + backdrop per 8 x 8 area	8	logos, plotting, scrolling pictures
Text40X	40 x 24, 6 x 8 characters	foreground/background per character	cursor	colour text screens
Text64	64 x 24, 8 x 8 characters, 768 with thirds	2 for the screen	cursor	terminals, editors, BASIC

TABLE 2-1 - EXTENDED DISPLAY MODES

2.3 Available Colours

The TMS9918B displays the 15 TMS9918A colours (SPPU004 Table 2-1). In extended modes each colour can be shown at four luminance levels through the palette, for up to 57 distinct colours; black stays black at every level.

Luminance bits (D2 D3)	Value	Output level above black
0 0	00h	full
0 1	10h	3/4
1 0	20h	1/2
1 1	30h	1/4

TABLE 2-2 - LUMINANCE LEVELS

2.4 Palettes

The palette has 16 entries: four palettes of four entries. A palette entry is one byte: luminance value (Table 2-2) plus the TMS colour number. Tiles, bitmap areas and sprites select one of the four palettes; their pixel values 1, 2 and 3 select the entries. Entry 0 of each palette is never displayed: pixel value 0 is the backdrop colour of R7 for tiles and bitmaps, and transparent for sprites. Graphics2Fat and the text modes do not use the palette.

Entry	Palette 0	Palette 1	Palette 2	Palette 3
Entries	0-3	4-7	8-11	12-15

TABLE 2-3 - PALETTE ENTRY NUMBERS

3. TALKING TO THE TMS9918B

All TMS9918A transfers of SPPU004 Section 4 remain valid. The TMS9918B adds three operations: the unlock command, palette writes and status register selection.

3.1 Unlocking the Extended Registers

After power-up the TMS9918B decodes register numbers like the TMS9918A (0 to 7). The unlock command is two consecutive writes of 5Ah to register 59. Both writes land in R3, so R3 must be written again afterwards.

Operation	MSB 0	1	2	3	4	5	6	LSB 7	Hex	MODE
Data write (byte 1)	0	1	0	1	1	0	1	0	5A	1
Register select (byte 2)	1	0	1	1	1	1	1	1	BB	1
Data write (byte 3)	0	1	0	1	1	0	1	0	5A	1
Register select (byte 4)	1	0	1	1	1	1	1	1	BB	1

TABLE 3-1 - UNLOCK COMMAND

NOTE

Once unlocked, register numbers 8 to 15 address the extended registers. Only a hardware RESET locks them again.

3.2 Detecting the TMS9918B

A program that uses extended functions must first verify the VDP. The sequence below is harmless on a TMS9918A, TMS99118, V9938 or F18A: every write that could land in another register is undone. Run it with interrupts disabled, because it reads the status register.

- Unlock (Section 3.1). Both key writes land in R3 on a locked device.
- Write 01h to R11 (XE). On other devices this lands in R3, like the key writes.
- Write 01h to R15 to select S1. On other devices this lands in R7.
- Read the status port twice. On a TMS9918B the second value AND 3Eh equals 18h.
- Write 00h to R15 to select S0 again, then restore R3 and R7: the probe aliases only those two.

EXAMPLE 3-1. Detecting the TMS9918B

```

DETECT: DI
        CALL UNLOCK          ; two writes of 5AH to register 59
        LD  A,01H            ; XE
        LD  B,11
        CALL WRREG
        LD  A,01H            ; select S1
        LD  B,15
        CALL WRREG
        IN  A,(VDPCTL)
        IN  A,(VDPCTL)      ; second read
        AND 3EH
        LD  C,A
        XOR  A               ; select S0
        LD  B,15
        CALL WRREG
        LD  A,(SHADOW3)     ; restore R3 and R7
        LD  B,3
        CALL WRREG
        LD  A,(SHADOW7)
        LD  B,7
        CALL WRREG
        LD  A,C
        CP  18H              ; Z = TMS9918B present
        EI
        RET

```

3.3 Register Aliasing While Locked

The TMS9918A decodes only the three low bits of a register number, and the TMS9918B does the same until it is unlocked. A write to register 8 therefore lands in R0, one to register 11 lands in R3, and so on. This is not a detail of the unlock command: it is the reason every probe in this guide restores a register afterwards, and the reason an extended program must detect the device before it writes anything above R7.

Extended register	Number AND 7	Register written while locked	What happens if it is not restored
R8 horizontal scroll	0	R0	the mode bit M3 and external video change: the display mode changes
R9 vertical scroll	1	R1	blanking, interrupt enable, mode bits and sprite size change
R10 line count	2	R2	the Name Table moves
R11 mode (XE, MX, IE1)	3	R3	the Colour Table moves
R12 screen (T64, MASK, locks)	4	R4	the Pattern Table moves
R13, R14 reserved	5, 6	R5, R6	the sprite tables move
R15 status select	7	R7	the text and backdrop colours change
R59 unlock	3	R3	the Colour Table moves, twice

TABLE 3-3 - WHAT AN EXTENDED REGISTER WRITE DOES ON A LOCKED DEVICE

Two consequences are worth keeping in mind. The first is that the detection sequence of Section 3.2 is safe only because it restores R3 and R7, the two registers its own writes alias. The second is more serious: a program that scrolls by writing R8 every frame would be writing R0 on a TMS9918A, changing the display mode sixty times a second. Detect first, and keep the extended writes behind the test.

NOTE

Aliasing disappears the moment the device is unlocked, and only a hardware RESET brings it back. Software that deliberately writes register numbers above 7 on a TMS9918A - a rare but legal practice - therefore behaves differently after the unlock. A program that hands control back to such software should write R11 = 00h, R12 = 00h, R8 = R9 = 00h and R15 = 00h, and request a hardware reset if it can.

3.4 Writing the Palette

With XE = 1, code 11 in the two most significant bits of the second control byte selects the palette. The first byte gives the entry number (0-15); every following write to the data port stores one entry and advances the entry number. The palette cannot be read back.

Operation	MSB 0	1	2	3	4	5	6	LSB 7	MODE
Entry number (byte 1)	0	0	0	0	E	E	E	E	1
Palette select (byte 2)	1	1	0	0	0	0	0	0	1
Entry data (byte 3, 4 ...)	0	0	L	L	C	C	C	C	0

TABLE 3-4 - WRITE TO PALETTE (E = ENTRY, L = LUMINANCE, C = COLOUR)

EXAMPLE 3-2. Loading all 16 entries

```

LOADPAL:                                ; HL -> 16 palette bytes
    XOR    A
    OUT    (VDPCTL),A    ; entry 0
    LD     A,0C0H
    OUT    (VDPCTL),A    ; code 11: palette
    LD     BC,1000H+VDPDAT
    OTIR
    RET

PALETTE:                                ; palette 0: -, white, grey, dark blue
    DB     00H,0FH,0EH,04H
    DB     00H,0BH,1BH,2BH ; palette 1: light yellow at full, 3/4, 1/2
    DB     00H,08H,09H,06H ; palette 2: reds
    DB     00H,03H,02H,0CH ; palette 3: greens

```

3.5 Reading the Status Registers

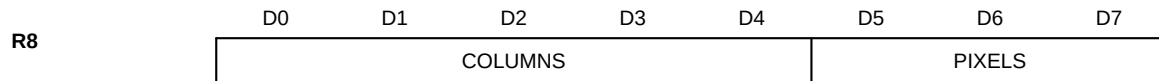
R15 selects the register returned by a status read: 01h selects S1, any other value S0. S1 contains the identification value 18h and, in its LSB (weight 01h), the scanline interrupt flag FL, which is cleared by the read. Leave R15 at 00h during normal operation so that frame interrupts are acknowledged as on the TMS9918A.

4. DESCRIPTION OF THE EXTENDED REGISTERS

4.1 Extended Write-Only Registers

Unused bits must be written as 0 to remain compatible with future devices.

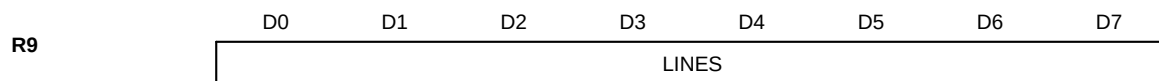
4.1.1 Register 8 (Horizontal Scroll)



The picture moves left as R8 increases: world X = screen X + R8. Bits 0-4 scroll by whole 8-pixel columns, bits 5-7 by pixels.

R8 is sampled once per line, at the first background access of that line: a value written later takes effect on the next line (Section 9.4).

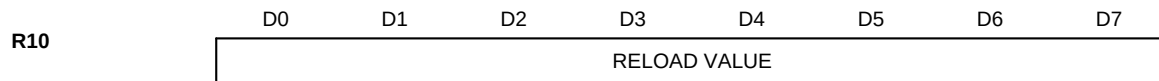
4.1.2 Register 9 (Vertical Scroll)



The picture moves up as R9 increases: world line = (screen line + R9) modulo 192. Values 192-255 are not useful.

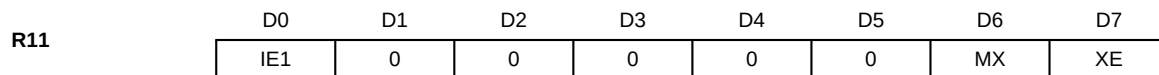
R9 is sampled once per line like R8, so a scanline interrupt can change either axis for the lines that follow.

4.1.3 Register 10 (Line Count)



Number of displayed lines between scanline interrupts minus 1 (Section 10).

4.1.4 Register 11 (Mode)

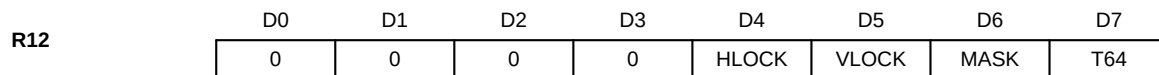


Bit 0 = IE1 (scanline interrupt enable). 0 disables, 1 enables the scanline interrupt.

Bit 6 = MX (extended modes). With XE = 1, selects the extended display modes (Section 5).

Bit 7 = XE (extended enable). 1 activates the palette, S1, R8-R15 and IE1. With XE = 0 the VDP behaves as a TMS9918A.

4.1.5 Register 12 (Screen)



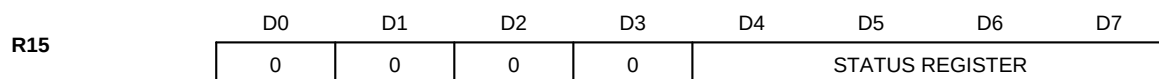
Bit 4 = HLOCK. 1: lines 0-15 are not scrolled horizontally (status bar).

Bit 5 = VLOCK. 1: columns 24-31 are not scrolled vertically (side panel).

Bit 6 = MASK. 1: the leftmost 8 pixels show the backdrop; use it with horizontal pixel scrolling.

Bit 7 = T64. 1: the extended text modes display 64 columns.

4.1.6 Register 15 (Status Select)



01h selects S1, any other value S0.

NOTE

R11 and R14 are reserved. R3 is not used in extended modes.

4.2 Status Register S1

	D0	D1	D2	D3	D4	D5	D6	D7
S1	0	0	0	1	1	0	0	FL

Bits 0-6 always read 0011000 (18h). Bit 7 = FL is set by the scanline counter and cleared when S1 is read.

5. INITIALIZING THE EXTENDED MODES

An extended mode is initialized like a TMS9918A mode (SPPU004 Section 6) plus four steps: unlock, set XE and MX in R11, set R12, and load the palette. The typical values below fit every table in 16K of VRAM; the resulting memory maps follow each table. Scroll registers R8 and R9 should be written as 00h.

The two-byte and four-byte entries of the extended tables are always fetched together. Tables must start at the addresses given by the registers; do not offset them by one byte.

5.1 Graphics1X Mode Initialization

Register	MSB LSB	Hex	Description
REG 0	00000000	00	Graphics1X (M3 = 0), no external video
REG 1	11000010	C2	16K, enable display, disable interrupt, 16 x 16 sprites, no magnification
REG 2	00001110	0E	Name/attribute table = 3800h (1536 bytes)
REG 3	XXXXXXXX	XX	Not used
REG 4	00000000	00	Pattern table = 0000h (512 patterns x 16 bytes)
REG 5	01111100	7C	Sprite attribute table = 3E00h
REG 6	00000101	05	Sprite pattern table = 2800h; BANK table = 3000h
REG 7	00000001	01	Backdrop colour = black
REG 11	00000011	03	Extended enable, extended modes
REG 12	00000010	02	MASK on (for horizontal scrolling)

TABLE 5-1 - GRAPHICS1X MODE INITIALIZATION

PATTERN TABLE	0000
UNUSED	2000
SPRITE PATTERNS	2800
SPRITE PATTERNS (BANK)	3000
NAME/ATTRIBUTE TABLE	3800
SPRITE ATTRIBUTES	3E00
UNUSED	3E80
	3FFF

FIGURE 5-1 - GRAPHICS1X MODE VRAM MEMORY MAP

5.2 Graphics2Fat Mode Initialization

Register	MSB LSB	Hex	Description
REG 0	00000010	02	Graphics2Fat (M3 = 1)
REG 1	11000010	C2	16K, enable display, disable interrupt, 16 x 16 sprites
REG 2	00001110	0E	Name/attribute table = 3800h
REG 3	XXXXXXXX	XX	Not used
REG 4	00000011	03	Pattern table = 0000h, one 256-pattern block per third
REG 5	01111100	7C	Sprite attribute table = 3E00h
REG 6	00000110	06	Sprite pattern table = 3000h (no room for BANK)
REG 7	00000001	01	Backdrop colour = black
REG 11	00000011	03	Extended enable, extended modes
REG 12	00000000	00	No mask, no locks

TABLE 5-2 - GRAPHICS2FAT MODE INITIALIZATION

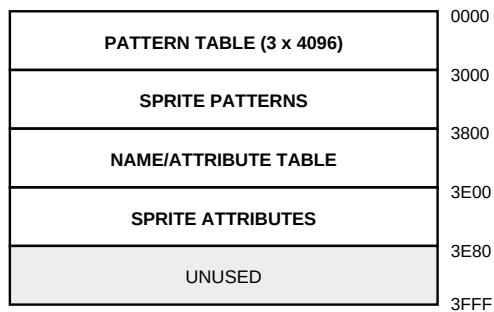


FIGURE 5-2 - GRAPHICS2FAT MODE VRAM MEMORY MAP

5.3 Bitmap Mode Initialization

Register	MSB LSB	Hex	Description
REG 0	00000000	00	Bitmap (M3 = 0); write 02h for BitmapQ
REG 1	11001010	CA	16K, enable display, disable interrupt, M2 = 1, 16 x 16 sprites
REG 2	00001110	0E	Palette map = 3800h (768 bytes)
REG 3	XXXXXXXX	XX	Not used
REG 4	00000000	00	Bitmap = 0000h (12288 bytes); write 03h for BitmapQ
REG 5	01110110	76	Sprite attribute table = 3B00h
REG 6	00000110	06	Sprite pattern table = 3000h
REG 7	00000001	01	Backdrop colour = black
REG 11	00000011	03	Extended enable, extended modes
REG 12	00000000	00	No mask, no locks

TABLE 5-3 - BITMAP MODE INITIALIZATION

BITMAP	0000
SPRITE PATTERNS	3000
PALETTE MAP	3800
SPRITE ATTRIBUTES	3B00
UNUSED	3B80
	3FFF

FIGURE 5-3 - BITMAP MODE VRAM MEMORY MAP

5.4 Text40X and Text64 Mode Initialization

Register	MSB LSB	Hex	Description
REG 0	00000000	00	Text40X/Text64 (M3 = 0)
REG 1	11010000	D0	16K, enable display, disable interrupt, M1 = 1, 8 x 8 cursor sprites
REG 2	00001100	0C	Character table = 3000h (Text40X 1920 bytes, Text64 1536 bytes)
REG 3	XXXXXXXX	XX	Not used
REG 4	00000000	00	Pattern table = 0000h
REG 5	01110110	76	Sprite attribute table = 3B00h (cursor sprites 0 and 1)
REG 6	00000001	01	Cursor pattern table = 0800h
REG 7	11110100	F4	White characters on dark blue
REG 11	00000011	03	Extended enable, extended modes
REG 12	00000001	01	T64 = 1 for 64 columns; 00h for Text40X

TABLE 5-4 - TEXT MODE INITIALIZATION

PATTERN TABLE	0000
CURSOR PATTERNS	0800
UNUSED	0900
CHARACTER TABLE	3000
UNUSED	3780
CURSOR ATTRIBUTES	3B00
UNUSED	3B08
	3FFF

FIGURE 5-4 - TEXT MODE VRAM MEMORY MAP

NOTE

Text64Q (M3 = 1) with 768 characters needs a 6144-byte pattern table: use R0 = 02h, R4 = 03h and move the cursor patterns with R6 = 04h (2000h).

EXAMPLE 5-1. Initializing Graphics1X

```
INITG1X:
    CALL DETECT
    RET  NZ          ; not a TMS9918B
    LD   HL,G1XREGS
    LD   B,10        ; ten register writes
INIT1:  LD   A,(HL)   ; value
    OUT  (VDPCTL),A
    INC  HL
    LD   A,(HL)      ; register number
    OR   80H
    OUT  (VDPCTL),A
    INC  HL
    DJNZ INIT1
    LD   HL,PALETTE
    JP   LOADPAL

G1XREGS:
    DB   00H,0, 0C2H,1, 0EH,2, 00H,4, 7CH,5
    DB   05H,6, 01H,7, 03H,11, 02H,12, 00H,8
```

6. CREATING PATTERNS FOR EXTENDED MODES

6.1 Two-Bit Tile Patterns

A Graphics1X pattern is drawn on an 8 x 8 grid where each square holds a value 0 to 3. Each row is stored as two bytes: the first byte holds the low bit of every value (stratum 0), the second byte the high bit (stratum 1). The 16 bytes of a pattern are stored row by row.

Row	Pixel values (left to right)	Stratum 0	Stratum 1
0	0 0 1 1 1 1 0 0	00111100 = 3C	00000000 = 00
1	0 1 2 2 2 2 1 0	01000010 = 42	00111100 = 3C
2	1 2 3 3 3 3 2 1	10111101 = BD	01111110 = 7E
3	1 2 3 0 0 3 2 1	10100101 = A5	01100110 = 66
4	1 2 3 0 0 3 2 1	10100101 = A5	01100110 = 66
5	1 2 3 3 3 3 2 1	10111101 = BD	01111110 = 7E
6	0 1 2 2 2 2 1 0	01000010 = 42	00111100 = 3C
7	0 0 1 1 1 1 0 0	00111100 = 3C	00000000 = 00

TABLE 6-1 - ENCODING A TWO-BIT PATTERN (A RING)

With palette 1 of Example 3-2 value 1 is light yellow at full luminance, 2 at 3/4 and 3 at 1/2; value 0 shows the backdrop. The pattern is stored as 3C 00 42 3C BD 7E A5 66 A5 66 BD 7E 42 3C 3C 00.

6.2 Graphics2Fat Patterns

A Graphics2Fat pattern row holds four dots, each two pixels wide, each with its own TMS colour number (0 = backdrop). The two low bits of every colour number go to the first byte, the two high bits to the second byte, dot 0 in bits 0 and 1 (MSB side).

Dot	0	1	2	3	Byte
Colour	F (white)	4 (dark blue)	0 (backdrop)	9 (light red)	
Binary	11 11	01 00	00 00	10 01	
Low bits -> byte 1	11	00	00	01	11000001 = C1
High bits -> byte 2	11	01	00	10	11010010 = D2

TABLE 6-2 - ENCODING A GRAPHICS2FAT ROW

6.3 Text Patterns

Text40X uses the 6 x 8 patterns of Text mode (SPPU004 Section 7.1.1): the two LSBs are not displayed. Text64 displays all 8 x 8 bits of every pattern; leave column 7 blank to separate characters. A Text64 font of 96 ASCII characters needs 768 bytes.

7. THE EXTENDED DISPLAY MODES

7.1 Graphics1X Mode

7.1.1 The Name/Attribute Table

Each of the 768 screen positions has two bytes: the pattern name and the attribute. The table is 1536 bytes long.

ATTRIBUTE	D0	D1	D2	D3	D4	D5	D6	D7
	0	0	0	0	NAME8	PRIOR	PALETTE	

NAME8 (08h) is the ninth bit of the name: names 000h-1FFh reach 512 patterns. PRIOR (04h) puts the tile in front of sprites wherever its pixel value is not 0. PALETTE (03h) selects palette 0-3.

EXAMPLE 7-1. To show pattern 12Ah with palette 2 in front of sprites at row 5, column 10 with R2 = 0Eh: the entry is at $3800h + 2 \times (5 \times 32 + 10) = 3954h$. Write 2Ah to 3954h and 0Eh (NAME8 + PRIOR + palette 2) to 3955h.

7.1.2 The Pattern Table

Pattern n starts at $(R4) \times 800h + 16 \times n$ and holds 16 bytes (Section 6.1). 512 patterns need 8192 bytes. Row r of pattern 12Ah with R4 = 00h is at $12A0h + 2 \times r$.

7.2 Graphics2Fat Mode

The name/attribute table is that of Graphics1X; only PRIOR is used in the attribute. The pattern table is split into three 4096-byte blocks, one per third of the screen, selected by R4 as in Graphics II (SPPU004 Section 8.3). With R4 = 03h the pattern of name n for the middle third starts at $1000h + 16 \times n$.

7.3 Bitmap and BitmapQ Modes

7.3.1 Plotting a Pixel

The bitmap has 64 bytes per line: two bytes (stratum 0, stratum 1) for every 8 pixels. For a pixel at X, Y with R4 = 00h:

$$\text{BYTE ADDRESS} = 64 \times Y + 2 \times \text{INT}(X/8)$$

The bit to change is selected by the remainder of X/8 with the table of SPPU004 Example 8-2 (0 = 80h ... 7 = 01h). Value 1 sets the bit in the first byte, value 2 in the second byte, value 3 in both, value 0 in neither.

EXAMPLE 7-2. Pixel X = 100, Y = 50, value 2: address $64 \times 50 + 2 \times 12 = 0C98h$, mask 08h. Clear bit 08h at 0C98h and set bit 08h at 0C99h.

7.3.2 The Palette Map

One byte per 8×8 area selects the palette of that area (bits 6-7): address $(R2) \times 400h + 32 \times \text{INT}(Y/8) + \text{INT}(X/8)$. The area of Example 7-2 is at $3800h + 32 \times 6 + 12 = 38CCh$.

7.3.3 BitmapQ

BitmapQ gives each third of the screen a 4096-byte block (64 lines $\times$ 64 bytes). With R4 = 03h the blocks are at 0000h, 1000h and 2000h and the line inside a block is Y AND 63. With R4 = 00h all thirds show the first block.

7.4 Text40X Mode

Each of the 960 positions has two bytes: the character name and a colour byte whose upper four bits give the character colour and lower four bits the background colour. A zero half uses the corresponding half of R7, so filling the colour bytes with 00h gives an ordinary Text mode screen.

EXAMPLE 7-3. Yellow "A" on dark red at row 2, column 5 with R2 = 0Ch: write 41h to 30AAh and B6h to 30ABh.

7.5 Text64 Mode

With T64 = 1 the screen has 64 columns of 8×8 characters and the character table has one byte per position, 1536 bytes. The whole screen uses the two colours of R7. With M3 = 1 (Text64Q) each third of the screen can use its own 256 characters, as in Graphics II.

EXAMPLE 7-4. Row 10, column 40 with R2 = 0Ch is at $3000h + 64 \times 10 + 40 = 32A8h$.

EXAMPLE 7-5. Printing a string in Text64

```
PRINT:      EX    DE,HL          ; DE = VRAM address, HL -> string, 0 ends
            CALL  SETWR
            EX    DE,HL
PRINT1:     LD    A,(HL)
            OR    A
            RET    Z
            OUT   (VDPDAT),A      ; 45 T per character: never loses data
            INC   HL
            JR    PRINT1
```

8. SPRITES IN EXTENDED MODES

The Sprite Attribute Table keeps its four-byte entries (SPPU004 Section 9.2). Only the fourth byte changes.

BYTE 3	D0	D1	D2	D3	D4	D5	D6	D7
	EC	XFINE	BANK	PAIR	PALETTE		ENTRY	

FIGURE 8-1 - SPRITE COLOUR BYTE IN EXTENDED MODES

8.1 Sprite Colour

PALETTE (0Ch) and ENTRY (03h) select the colour: entry 1-3 of palette 0-3. Entry 0 makes the sprite invisible, which is useful for blinking: the sprite still takes part in coincidence checking.

8.2 BANK

With BANK (20h) set, the pattern of the sprite is taken from the 2048 bytes following the table located by R6. Names 00h-FFh of both tables give 512 patterns: store mirrored frames in the second table instead of flipping sprites in software.

8.3 Sprite Pairs

PAIR (10h) is valid on odd sprites. Sprite $2k+1$ with PAIR set becomes the second bit plane of sprite $2k$: where only sprite $2k$ has a pixel the colour is entry 1, only sprite $2k+1$ entry 2, both entry 3, all from the palette of sprite $2k$. The two sprites keep separate positions and names, so they can be placed exactly on top of each other for a three-colour sprite, or offset for large figures. Their overlap never sets the coincidence flag.

EXAMPLE 8-1. A three-colour 16 x 16 sprite from sprites 0 and 1

```
; patterns 00H (plane 0) and 04H (plane 1) are loaded at 2800H
SAT:  DB  60H,78H,00H,05H ; sprite 0: Y, X, name 00H, palette 1 entry 1
      DB  60H,78H,04H,10H ; sprite 1: same Y, X, name 04H, PAIR
      DB  0D0H             ; end of table
```

8.4 Sprites per Line

Eight sprites can share a horizontal line in Graphics1X, Graphics2Fat and Bitmap modes. The TMS9918B selects the sprites of two lines at a time, so a sprite that ends on the first line of a pair still counts on the second one. When a ninth sprite is found, 5S is set and its number is loaded into S0; sprites of higher number are not displayed on those lines. A pair uses two of the eight sprites.

8.5 Sprites in the Text Modes

Text40X, Text40XQ, Text64 and Text64Q display sprites 0 and 1 and ignore sprites 2 to 31. Two is what the memory access sequence of a text line affords: the VDP reads the vertical position of every sprite before it can decide which ones belong to a line, and a text line has only sixteen spare cycles. In every other respect these two behave like the sprites of the graphics modes - same table, same patterns, same palette - and PAIR still combines them into one three-colour object.

Their patterns are not stretched. In a 64-column mode the picture is 512 pixels wide and a sprite pixel is one of those pixels, not two, so an 8x8 sprite covers exactly one character cell and a 16x16 sprite covers two. A VDP that already shifts 512 pixels per line has no reason to double them again, and this is what makes a cursor the size of the character it sits on.

8.5.1 A Blinking Cursor

A text cursor sits on a character boundary, so its position is a multiple of the cell width and never needs anything finer. Set X once and leave it; to blink, alternate the ENTRY bits of the colour byte between a colour and 0, which makes the sprite invisible. One byte every thirty frames, and no pattern is rewritten.

8.5.2 Why the X Byte Is Not Enough at 64 Columns

The X byte holds 0 to 255. A 40-column screen is 256 pixels wide, so the byte reaches every pixel of it. A 64-column screen is 512 pixels wide and the byte reaches half of them. Something has to supply the missing bit.

In a 512-pixel mode the TMS9918B reads the X byte as a count of two-pixel steps, so a sprite can sit on any even pixel: $X = 4 * \text{column}$ places it on a character boundary. The odd pixels come from XFINE, bit 1 of the colour byte, which is unused in every other mode:

$$\text{SCREEN } X = 2 \times X + \text{XFINE}$$

Screen column	X byte	XFINE
0, the first character	0	0
1	0	1
8, the second character	4	0
201	100	1
510	255	0
511	255	1

TABLE 8-3 - SPRITE POSITIONS AT 64 COLUMNS

NOTE

XFINE applies to sprites 0 and 1, the only ones a text mode displays, and only in a 512-pixel mode. Everywhere else the bit is reserved and must be written as 0, so that it remains available.

8.5.3 Smooth Movement

A cursor never needs XFINE. Anything that moves does. An object crossing a 512-pixel line in two-pixel steps covers it in 256 steps instead of 512, and at any speed slow enough to be followed by the eye the movement is visibly stepped - a pointer, a selection bar sliding along a menu, a marker following a waveform. The same object with XFINE moves one pixel at a time.

This is the whole purpose of the bit. It is not tied to a pointing device: it is what makes horizontal movement at 64 columns as smooth as it is at 40, where the X byte already addresses every pixel.

8.5.4 Why the Bit Is the Low One

The obvious place for a missing bit is the top: keep X as a count of pixels and let the spare bit carry the value 256. The TMS9918B does the opposite, and the reason is what happens while software is writing.

A position lives in two bytes of the Sprite Attribute Table and the VDP reads that table once per line, so a program that moves a sprite has a window in which one byte is new and the other still old. With the bit at the top, an update caught in that window puts the sprite 256 pixels from where it belongs for one frame, which the eye reads as a flash on the other side of the screen. With the bit at the bottom the same accident leaves it one pixel out, which nobody sees.

The two bytes are not two halves of one number. X places the sprite to within a pixel and XFINE moves it by that pixel, exactly as the column bits and the fine bits of R8 do for the playfield. Writing X first and XFINE second is therefore the natural order and needs no protection: after the first write the sprite is already almost where it belongs.

There is a second advantage. Software that knows nothing about XFINE writes 0 there and gets exactly the behaviour it expects, so the bit refines a coordinate instead of redefining it.

NOTE

Two alternatives were rejected. Taking the low bit from the sprite number - even sprites on even pixels, odd sprites on odd ones - costs less silicon but ties a position to a table entry: moving an object from one sprite to another would move it by a pixel, and the two bit planes of a PAIR would land on different columns. Leaving the coordinate at two-pixel steps costs nothing and is enough for a cursor, but not for anything that moves.

8.5.5 Early Clock at 64 Columns

EC subtracts 32 from the coordinate, and the coordinate is the coarse one, so in a 512-pixel mode a sprite moves left by 64 pixels rather than 32. The purpose is unchanged, letting a sprite enter from the left edge, and the distance is the width of eight characters.

8.5.6 Writing a Position

The four bytes of an entry are adjacent and the address register increments after every write, so one address setup sends the whole entry: Y, X, pattern name and colour byte. That is the cheapest way to move an object and it leaves the shortest window, because X and XFINE are two writes apart rather than a second address setup apart.

The window closes entirely if the table is written during vertical blanking, where the VDP reads nothing and 4300 microseconds are available, room for all 32 entries. A program that moves an object from a scanline interrupt does not have that luxury and does not need it: the worst it can suffer is the one-pixel offset described above.

EXAMPLE 8-2. Moving a pointer to any of the 512 columns

; HL = column 0..511, B = raw Y, C = pattern name, A = colour byte with XFINE
; clear. The four bytes of the entry are written in one go: the address
; register increments, so one setup covers them all.

```
POINTER:
    LD    (COLSHADOW),A
    SRL   H
    RR    L           ; HL = column / 2 = the X byte
    LD    A,L
    LD    (XSHADOW),A
    LD    HL,SAT      ; the sprite 0 entry
    CALL SETWR
    LD    A,B
    OUT   (VDPDAT),A  ; Y
    LD    A,(XSHADOW)
    OUT   (VDPDAT),A  ; X, two-pixel steps
    LD    A,C
    OUT   (VDPDAT),A  ; pattern name
    LD    A,E         ; E holds the column LSB saved by the caller
    AND   01H
    RRCA
    RRCA             ; bit 0 becomes XFINE, weight 40H
    LD    HL,COLSHADOW
    OR    (HL)
    OUT   (VDPDAT),A  ; colour byte with XFINE
    RET
```

EXAMPLE 8-3. Blinking the cursor from the frame interrupt

```
BLINK:  LD    HL,BLINKCNT
        DEC   (HL)
        RET   NZ
        LD    (HL),30
        LD    HL,3B03H    ; colour byte of sprite 0
        CALL SETWR
        LD    A,(CURCOL)
        XOR   01H        ; toggle entry 1 <-> 0
        LD    (CURCOL),A
        OUT   (VDPDAT),A
        RET
```

9. SCROLLING

SPPU004 Section 10.1 scrolls a TMS9918A screen by rewriting the Name Table, eight pixels at a time and 768 bytes per step. The TMS9918B scrolls by single pixels with two registers and no VRAM write at all. This section explains what the two registers do, what the three bits of R12 are for, when a written value becomes visible, and how to keep scrolling past the edge of the 32x24 world.

9.1 The World and the Screen

Think of the Name Table as a world of 32 by 24 cells that wraps in both directions, and of the screen as a window on it. The two registers move the window:

Register	Effect	Wrap
R8, horizontal	world X = screen X + R8. Bits 0-4 are whole 8-pixel columns, bits 5-7 are pixels.	every 256 pixels, that is 32 columns
R9, vertical	world line = (screen line + R9) modulo 192.	every 192 lines, that is 24 rows

TABLE 9-1 - THE SCROLL REGISTERS

Increasing a register moves the picture left or up, which is the same as moving the window right or down. Both are unsigned: to scroll the other way, decrement. The registers never change what is stored in VRAM, so a screen can scroll for as long as the program wants while the CPU does something else.

Scrolling applies to Graphics1X, Graphics2Fat, Bitmap and BitmapQ. The extended text modes accept R9 only: the character cell is six or eight pixels wide and the columns are fetched in a fixed order, so there is no horizontal scrolling at 40 or 64 columns.

9.2 What the VDP Does With Them

The memory access sequence never changes: the 32 cells of a line are fetched in the same cycles whatever the scroll value is. Only two things change.

The address. Active cell c fetches world column $(c + 1 + R8 / 8)$ AND 31, and every address of the line - name, attribute, pattern row and, with M3, the third - is computed from the world line, so banked content scrolls as a whole.

The output tap. The pixels of a cell go through a shift register; the low three bits of R8 select which pixel of it appears first. That is what makes the movement smooth instead of jumping by eight pixels.

Because the fetch starts one column early, a partial column always appears in the leftmost eight pixels when R8 is not a multiple of eight. Section 9.3 is about hiding it.

9.3 MASK, HLOCK and VLOCK

Bit	Weight	Effect	Used for
MASK	02h	The leftmost eight pixels always show the backdrop of R7, sprites included.	hiding the partial column; set it whenever R8 is used
HLOCK	08h	Lines 0-15, that is character rows 0 and 1, ignore R8.	a score bar that stays still while the playfield scrolls
VLOCK	04h	Columns 24-31 ignore R9.	a side panel that stays still while the playfield scrolls
T64	01h	Not a scrolling bit: 64-column text.	-

TABLE 9-2 - REGISTER 12

NOTE

A locked area ignores one axis, not both: with HLOCK the status bar still moves vertically, and with VLOCK the side panel still moves horizontally. A panel that must never move needs both bits, or a screen that scrolls on one axis only.

9.4 When a Written Value Becomes Visible

Both registers are sampled once per line, at the start of the horizontal blanking that precedes the line, together with the sprites of that line. A value written after that moment takes effect on the next line.

This is a guarantee, not a limitation. It means that a cell can never take its name from one world position and its pattern from another, whatever the CPU does; that the same program produces the same picture on every implementation; and that a scanline interrupt can change either axis for the lines that follow, which is what Section 10 is about. The TMS9918B has no mid-line effects at all: the smallest unit that can change is one line.

NOTE

The Sega VDP of 1985 latches the vertical scroll once per frame and ignores writes until the next one, so raster effects there can change the horizontal axis only. The TMS9918B samples both axes per line, which is why the split screens of Section 10.4 can move vertically as well.

9.5 Scrolling Beyond One Screen

The world is only 32 by 24 cells, so after 256 pixels the picture repeats. To scroll indefinitely, rewrite the column - or the row - that is about to enter the screen, one every eight pixels of movement. The column to rewrite is the one the window is leaving, because the world wraps:

```
; Called once per frame. Scrolls right by one pixel and refills a column
; every eight pixels. WORLD_X is the 16-bit position in the world.
SCROLL_STEP:
    ld hl,(WORLD_X)
    inc hl
    ld (WORLD_X),hl
    ld a,l
    ld b,8                ; R8 = low byte of the world position
    call WRREG
    and 7
    ret nz                ; only every eight pixels
    ld a,l
    rrca
    rrca
    rrca
    and 31                ; column entering on the right
    add a,31
    and 31                ; the column leaving on the left holds it
    jp REFILL_COLUMN      ; writes 24 entries of two bytes
```

Twenty-four two-byte entries are 48 bytes: at 21 T-states each in a tile mode this is about 1000 T-states, well inside a frame. The same method applies to R9 with rows instead of columns.

9.6 Sprites Are Not Scrolled

The scroll registers move the playfield only. A sprite is placed in screen coordinates, so a program keeps its objects in world coordinates and subtracts the scroll position when it writes the Sprite Attribute Table:

```
; DE = world X of the object, HL = world Y. Produces the SAT bytes.
    ld a,e
    ld hl,(WORLD_X)
    sub l                ; screen X = world X - scroll X
    ld (SAT_X),a
```

This is the same arrangement every scrolling machine of the period used, and the reason the hardware cursor of the text modes is not affected by R9 either.

10. THE SCANLINE INTERRUPT

The frame interrupt of the TMS9918A tells a program that the picture is finished. The scanline interrupt tells it that a chosen line is finished, while the picture is still being drawn: that is what turns one playfield into several bands with different scroll positions, palettes or backdrops.

10.1 The Line Counter

R10 holds a reload value. The counter is loaded from R10 at the first line of the picture and whenever R10 is written; it is decremented at the end of every displayed line, and when it would go below zero the flag FL is set and the counter is reloaded.

R10	Interrupt at the end of lines
0	every line: 0, 1, 2, ... 191
7	every eight lines: 7, 15, 23, ... 191
47	47, 95, 143, 191: the screen in four equal bands
95	95 and 191: two halves
192 or more	never, because only lines 0-191 are counted

TABLE 10-1 - TYPICAL LINE COUNTS

NOTE

Writing R10 reloads the counter at once, so a handler can change the distance to the next interrupt: the bands of a screen do not have to be equally spaced.

10.2 Enabling and Acknowledging

IE1 in R11 connects the flag to the INT pin, which is shared with the frame interrupt of R1. A handler must therefore find out which of the two occurred, and clear both flags before returning:

Step	Why
Write 01h to R15	select S1
Read the status port	S1: bit 7 (weight 01h) is FL; the read clears it
Write 00h to R15	select S0 again, at once
Read the status port	S0: bit 0 (weight 80h) is the frame flag; the read clears it
Act on the flags	FL: paint the next band. Frame flag: start the frame again.

TABLE 10-2 - ACKNOWLEDGING BOTH INTERRUPTS

Leaving R15 at 00h outside the handler is a convention, not a rule of the device: it keeps the status port showing S0 for any code that expects the TMS9918A.

10.3 What a Handler Can Change in Time

The interrupt is raised at the end of line N. The registers of line N+1 are sampled during the horizontal blanking that has just begun, so a short handler that writes R8, R9, R7 or a palette entry immediately is in time for the very next line; a longer one takes effect one or two lines later. A band boundary is therefore stable to within a line or two, which is why bands are usually placed where a line of colour separates them.

The safest arrangement is to prepare everything before the interrupt and to write only registers inside it: no VRAM addressing, no long loops, no two-byte control transfers that another interrupt could break.

10.4 Four Things a Band Can Do

Effect	What the handler writes	Result
Parallax	R8 with a different value per band	a sky that moves slower than the ground
Fixed status bar	R8 = 0 for the first band, the scroll value for the rest	the same as HLOCK, but with the boundary where the program wants it
Vertical split	R9 with a different value per band	two halves of the world on one screen
Colour change	R7 or a palette entry	a coloured sky, a different backdrop per band, more than 16 colours on screen

TABLE 10-3 - USES OF THE SCANLINE INTERRUPT

EXAMPLE 10-1. Two bands with different horizontal speeds

```

; R10 = 95: the interrupt fires at the end of lines 95 and 191.
; BAND holds which band the handler is about to prepare.
IRQ:  PUSH AF
      PUSH BC
      LD  A,01H
      LD  B,15
      CALL WRREG          ; select S1
      IN  A,(VDPCTL)      ; read S1: bit 7 is FL
      LD  C,A
      XOR  A
      LD  B,15
      CALL WRREG          ; back to S0
      IN  A,(VDPCTL)      ; read S0: clears the frame flag
      BIT  0,C            ; weight 01H
      JR  Z,FRAME
      LD  A,(SCROLL_FAR)  ; lower band: the ground
      LD  B,8
      CALL WRREG
      JR  DONE
FRAME: LD  A,(SCROLL_NEAR) ; upper band: the sky
      LD  B,8
      CALL WRREG
      LD  HL,SCROLL_NEAR  ; move both layers for the next frame
      INC (HL)
      LD  HL,SCROLL_FAR
      INC (HL)
      INC (HL)
DONE:  POP  BC
      POP  AF
      EI
      RETI

```

11. OTHER PROGRAMMING TIPS

11.1 Palette Effects

Palette writes never wait for a memory access window, so they can be done at any time. Fading in and out is done by stepping the luminance value of every entry from 30h to 00h and back, one step every few frames. Colour cycling is done by rewriting the three entries of a palette in rotation: animated water, conveyor belts and flashing lights need no pattern or name changes.

11.2 Loading VRAM Quickly

The CPU can transfer data to VRAM without losing bytes when each transfer takes at least the total time of Appendix B for the mode. The fastest safe loops are:

Mode	Fastest safe loop	T-states per byte
Graphics I, II (TMS9918A modes)	loop of 29 T or more, or OTIR during vertical blanking	29
Graphics1X, Graphics2Fat	OTIR	21
Bitmap, BitmapQ	OUTI chain	16
Text40X, Text64	OUTI chain	16
Any mode, vertical blanking or display blanked	OUTI chain	16

TABLE 11-1 - FAST VRAM TRANSFERS

APPENDIX A - REGISTER MAP

Every write-only register and every status register of the TMS9918B. Registers 0 to 7 keep their TMS9918A meaning; registers 8 to 15 exist after the unlock command of Section 3.1. Bit 0 is the MSB, and the weight column gives the value to OR into the register.

Register	Name	Bits	Weight	Function
R0	Mode 1	D6 M3	02h	Mode bit 3; with MX selects the Q variants
		D7 EXTVID	01h	External VDP plane
R1	Mode 2	D0 4/16K	80h	Must be 1: the extended modes need 16K
		D1 BL	40h	Display enable
		D2 IE	20h	Frame interrupt enable
		D3 M1, D4 M2	10h, 08h	Mode bits 1 and 2
		D6 SIZE, D7 MAG	02h, 01h	Sprite size and magnification
R2	Name base	D4-D7	-	Name table at R2 x 400h; entries are two bytes in the extended modes
R3	Colour base	all	-	TMS9918A colour table; not used in the extended modes
R4	Pattern base	D5-D7	-	Pattern or bitmap table; D6-D7 are the third-banking mask with M3
R5	SAT base	D1-D7	-	Sprite attribute table at R5 x 80h
R6	Sprite patterns	D5-D7	-	Sprite pattern table at R6 x 800h; BANK adds 800h
R7	Colours	D0-D3	-	Text foreground; also the fallback ink of Text40X
		D4-D7	-	Backdrop, and the paper of the text modes
R8	H scroll	D0-D4	-	Horizontal scroll, whole columns
		D5-D7	-	Horizontal scroll, pixels. Sampled once per line
R9	V scroll	all	-	Vertical scroll, 0-191. Sampled once per line
R10	Line count	all	-	Lines between scanline interrupts, minus one
R11	Mode	D0 IE1	80h	Scanline interrupt enable
		D6 MX	02h	Extended display modes; requires XE
		D7 XE	01h	Extended enable: palette, S1, R8-R15, IE1
R12	Screen	D4 HLOCK	08h	Lines 0-15 are not scrolled horizontally
		D5 VLOCK	04h	Columns 24-31 are not scrolled vertically
		D6 MASK	02h	The leftmost eight pixels show the backdrop
		D7 T64	01h	64-column text
R13, R14	reserved	-	-	Write 00h
R15	Status select	D4-D7	-	01h selects S1, any other value S0
R59	Unlock	all	-	Two consecutive writes of 5Ah unlock R8-R15
S0	Status	D0 F	80h	Frame interrupt flag; cleared on read
		D1 5S	40h	Ninth sprite on a line in the extended modes
		D2 C	20h	Sprite coincidence
		D3-D7	1Fh	Number of the reported sprite
S1	Status	D0-D6	3Eh mask	Identification: the masked value is 18h
		D7 FL	01h	Scanline interrupt flag; cleared on read

TABLE A-1 - COMPLETE REGISTER MAP

APPENDIX B - CPU TO VDP ACCESS TIMES

Condition	Mode	VDP delay	Time waiting for an access window	Total time
Active display area	Text	2 us	0 - 1.1 us	2 - 3.2 us
Active display area	Graphics I, II	2 us	0 - 6.0 us	2 - 8.0 us
Active display area	Multicolor	2 us	0 - 1.5 us	2 - 3.5 us
Any time	Graphics1X, Graphics2Fat	2 us	0 - 3.0 us	2 - 5.0 us
Any time	Bitmap, BitmapQ	2 us	0 - 1.9 us	2 - 4.0 us
Any time	Text40X	2 us	0 - 2.2 us	2 - 4.3 us
Any time	Text64	2 us	0 - 1.6 us	2 - 3.7 us
4300 us after vertical interrupt	All	2 us	0 us	2 us
Register 1 blank bit 0	All	2 us	0 us	2 us
Palette write	Extended	-	0 us	immediate

TABLE B-1 - CPU TO VDP ACCESS TIMES

APPENDIX C - ADDRESS LOCATION TABLES

Address type	Mode	Address	Worked example
Name/attribute	Graphics1X, Graphics2Fat	$(R2) \times 400h + 2 \times (ROW \times 32 + COLUMN)$	R2 = 0Eh, row 5, column 10: 3954h/3955h
Pattern row	Graphics1X	$(R4) \times 800h + 16 \times NAME9 + 2 \times PATTERN ROW$	R4 = 00h, name 12Ah, row 3: 12A6h/12A7h
Pattern row	Graphics2Fat	third block + $16 \times NAME + 2 \times PATTERN ROW$	R4 = 03h, middle third, name 20h, row 0: 1200h
Bitmap	Bitmap	$(R4) \times 800h + 64 \times Y + 2 \times INT(X/8)$	X 100, Y 50: 0C98h/0C99h
Bitmap	BitmapQ	third block + $64 \times (Y AND 63) + 2 \times INT(X/8)$	R4 = 03h, X 0, Y 100: 1900h
Palette map	Bitmap, BitmapQ	$(R2) \times 400h + 32 \times INT(Y/8) + INT(X/8)$	X 100, Y 50: 38CCh
Character/colour	Text40X	$(R2) \times 400h + 2 \times (ROW \times 40 + COLUMN)$	R2 = 0Ch, row 2, column 5: 30AAh/30ABh
Character	Text64	$(R2) \times 400h + 64 \times ROW + COLUMN$	R2 = 0Ch, row 10, column 40: 32A8h
Sprite pattern	extended	$(R6) \times 800h + (BANK) \times 800h + 8 \times NAME$	R6 = 05h, name 10h, BANK: 3080h

TABLE C-1 - EXTENDED ADDRESS LOCATIONS

APPENDIX D - Z80 SUPPORT ROUTINES

The examples of this supplement use the routines below. SHADOW0 and SHADOW7 hold the last values written to R0 and R7.

```

VDPDAT EQU 0BEH          ; MODE = 0
VDPCTL EQU 0BFH          ; MODE = 1

WRREG:                    ; A = value, B = register number 0-63
    OUT (VDPCTL),A
    LD  A,B
    OR  80H
    OUT (VDPCTL),A
    RET

SETWR:                    ; HL = VRAM address for writing
    LD  A,L
    OUT (VDPCTL),A
    LD  A,H
    AND 3FH
    OR  40H
    OUT (VDPCTL),A
    RET

SETRD:                    ; HL = VRAM address for reading
    LD  A,L
    OUT (VDPCTL),A
    LD  A,H
    AND 3FH
    OUT (VDPCTL),A
    RET

UNLOCK:                   ; unlock registers 8-15, restore R7
    LD  A,5AH
    LD  B,63
    CALL WRREG
    LD  A,5AH
    CALL WRREG            ; B is still 63
    LD  A,(SHADOW7)
    LD  B,7
    JP  WRREG

LDIRVM:                   ; copy BC bytes from HL to VRAM address DE
    EX  DE,HL
    CALL SETWR
    EX  DE,HL
LDIRV1: LD  A,(HL)
    OUT (VDPDAT),A
    INC HL
    DEC BC
    LD  A,B
    OR  C
    JR  NZ,LDIRV1        ; 50 T per byte: safe in every mode
    RET

```